Coding Practice Set - 02

Problem 1: Print Numbers from 1 to N

Problem Statement: Write a recursive function that takes an integer n as an argument and prints all numbers from 1 to n in ascending order.

> **Base Case Hint:** Think about the absolute smallest positive integer bound where the function should stop execution (e.g., when n reaches 0).
>
> **Recursive Case Hint:** You want to print numbers in ascending order. Should you print the current value of n *before* the function calls itself (winding phase), or *after* the function returns from deeper calls (unwinding phase)?

Problem 2: Print Numbers from N to 1

Problem Statement: Write a recursive function that takes an integer n as an argument and prints all numbers from n down to 1 in descending order.

> **Base Case Hint:** Determine the termination condition to prevent the function from processing numbers less than 1.
>
> **Recursive Case Hint:** To print in descending order, you need to output the current value of n immediately during the winding phase before moving on to the next smaller sub-problem (n - 1).

Problem 3: Count the Digits of a Number

Problem Statement: Write a recursive function to count the total number of digits in a given integer n.

> **Base Case Hint:** At what point is a number so small that you instantly know it only contains a single digit? (Think about numbers less than 10).
>
> **Recursive Case Hint:** You can chop off the last digit of a number using integer division (n / 10). Every time you successfully strip away a digit, you add 1 to the count of the remaining sub-problem.

Problem 4: Find the N^th Fibonacci Number

Problem Statement: The Fibonacci series starts with 0 and 1, and each subsequent number is the sum of the two preceding ones (0, 1, 1, 2, 3, 5, 8, ……). Write a recursive function to find the N^th Fibonacci number.

Base Case Hint: This problem requires two distinct base cases because the calculation depends on the two previous numbers. What are the known values when n is exactly 0 or 1?

Recursive Case Hint: Formulate the expression where the function breaks the problem down into the sum of its two immediate mathematical predecessors: f(n) = f(n-1) + f(n-2).

Problem 5: Power Calculation (X^Y)

Problem Statement: Write a recursive function to calculate the value of a base number X raised to the power of an exponent Y (i.e., X^Y).

Base Case Hint: What is the mathematically predefined result of any non-zero number raised to the power of 0?

Recursive Case Hint: Express X^Y by isolating one unit of the base X and multiplying it by a smaller exponent sub-problem: power(X, Y) = X * power(X, Y-1).